

PROJECT REPORT

CineMatch

Simple Movie Recommendation System

A console-based Python project

Student Name	Aditya Prasad
Registration No.	26BAI10330
Programme	B.Tech CSE (AI and ML)
University	VIT Bhopal University
Course Name	CSE1021
Faculty / Instructor	Balaji A

Table of Contents

Table of Contents	2
2. Introduction	3
3. Problem Statement	3
4. Functional Requirements	3
5. Non-functional Requirements	4
6. System Architecture	4
7. Design Diagrams	6
7.1 Use Case Diagram	6
7.2 Workflow Diagram	7
7.3 Sequence Diagram	8
7.4 Class / Component Diagram	9
7.5 ER Diagram	9
8. Design Decisions and Rationale	9
9. Implementation Details	10
10. Screenshots / Results	11
11. Testing Approach	13
12. Challenges Faced	14
13. Learnings and Key Takeaways	14
14. Future Enhancements	14
15. References	15

2. Introduction

Choosing what to watch has become harder than watching itself. Streaming platforms such as Netflix, Prime Video and Disney+ host thousands of titles, and viewers often spend a long time scrolling before settling on a movie. **CineMatch** is a small recommendation system that removes this friction. The user answers three simple questions - the genre they feel like watching, the film industry (Hollywood or Bollywood) and the OTT platform they use - and the program instantly lists matching movies, best-rated first.

The system is written in Python and runs in the console. It contains a built-in collection of 100 popular movies (50 Hollywood and 50 Bollywood). Every movie is stored as a tuple of five fields: name, genre, industry, IMDb rating and OTT platform. The project intentionally uses only fundamental Python concepts - lists, tuples, loops, conditions, functions and user input - so that its logic is easy to follow, test and extend.

Objectives

- Build a working recommendation tool using only basic Python features.
- Let users filter movies by genre, industry and OTT platform, with an option to skip any filter.
- Present the results in a clear, ranked format based on IMDb rating.
- Document the design with standard diagrams and a proper testing approach.

3. Problem Statement

With so many titles spread across multiple streaming platforms, viewers waste time browsing, comparing ratings on one site and checking availability on another. There is a need for a simple tool that lets a user narrow down movies according to what they care about - **genre, industry and OTT platform** - and then presents the highest-rated options first.

Scope. The project is a console application with a fixed, built-in dataset. It filters, sorts and displays movies. It does not include a graphical or web interface, live internet data, user accounts, watch history or search by title, actor or director.

Target users. Movie lovers who want a quick suggestion, OTT subscribers who only want titles on the platform they already pay for, groups deciding what to watch together, and beginner Python learners who want a practical example.

4. Functional Requirements

The functional requirements describe what the system must do.

ID	Requirement
FR1	The system shall display a welcome banner and the list of available genres when it starts.
FR2	The system shall accept a genre from the user (Action, Comedy, Crime, Drama, Fantasy, Horror, Romance, Sci-Fi, Thriller, War, Western) or the keyword Any .
FR3	The system shall accept an industry (Hollywood or Bollywood) or Any .
FR4	The system shall accept an OTT platform (Netflix, Prime Video or Disney+) or Any .

ID	Requirement
FR5	The system shall ignore letter case and leading or trailing spaces in all inputs.
FR6	The system shall select only those movies that satisfy all three chosen filters.
FR7	The system shall sort the selected movies by IMDb rating from highest to lowest.
FR8	The system shall display, for each recommended movie, its number, name, genre, industry, IMDb rating and OTT platform.
FR9	The system shall show a clear message when no movie matches the user's choices.
FR10	The system shall show a closing message when the program ends.

5. Non-functional Requirements

The non-functional requirements describe how well the system should behave.

Attribute	Requirement
Usability	Prompts must be simple and self-explanatory; a first-time user should get results without reading any manual.
Performance	Filtering and sorting of the dataset shall complete effectively instantly (well under one second) for the built-in movie list.
Reliability	The program shall not crash on unexpected text input; unmatched choices lead to a friendly 'No movies found' message.
Portability	The program shall run on any operating system with Python 3 installed, without extra libraries.
Maintainability	Code shall be split into small functions with comments so that movies or features can be added easily.
Simplicity	Only basic Python constructs shall be used so that the code is easy to read and understand.
Accuracy	Displayed details shall exactly match the stored data of each movie.

6. System Architecture

CineMatch follows a simple three-layer structure. The **presentation layer** handles all interaction with the user through the console. The **application logic layer** cleans the input, filters the movies and sorts the result. The **data layer** is an in-memory list of movie tuples. Because the whole program runs in a single process and needs no database or network, the architecture stays small and easy to test.

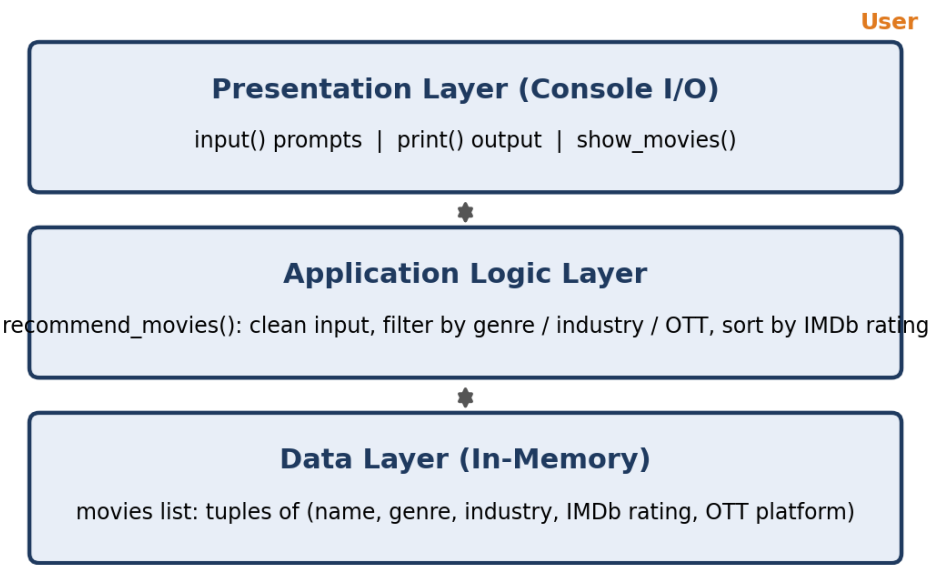


Figure 1: Layered architecture of CineMatch

Data model. Each movie record is a tuple with the following fields:

Index	Field	Type	Example
0	Movie name	String	Inception
1	Genre	String	Sci-Fi
2	Industry	String	Hollywood
3	IMDb rating	Float	8.8
4	OTT platform	String	Prime Video

7. Design Diagrams

This section presents the design of CineMatch using standard UML-style diagrams.

7.1 Use Case Diagram

The only actor is the **User**. The user enters the genre, industry and OTT platform and asks for recommendations. Viewing recommendations internally includes filtering the movies and sorting them by IMDb rating.

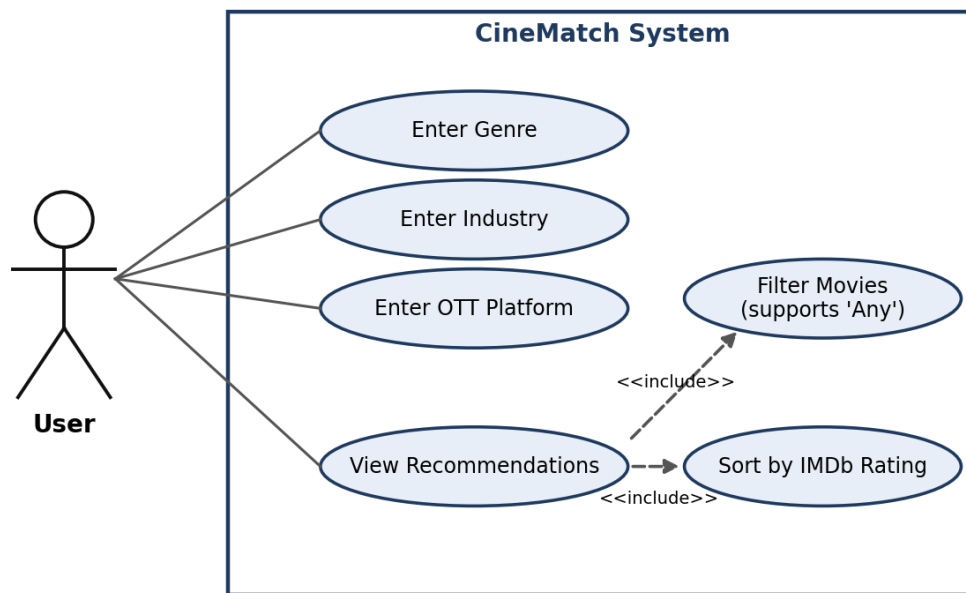


Figure 2: Use case diagram

7.2 Workflow Diagram

The flowchart shows the complete flow of the program, from the welcome banner to the final output, including the loop that checks every movie against the three filters.

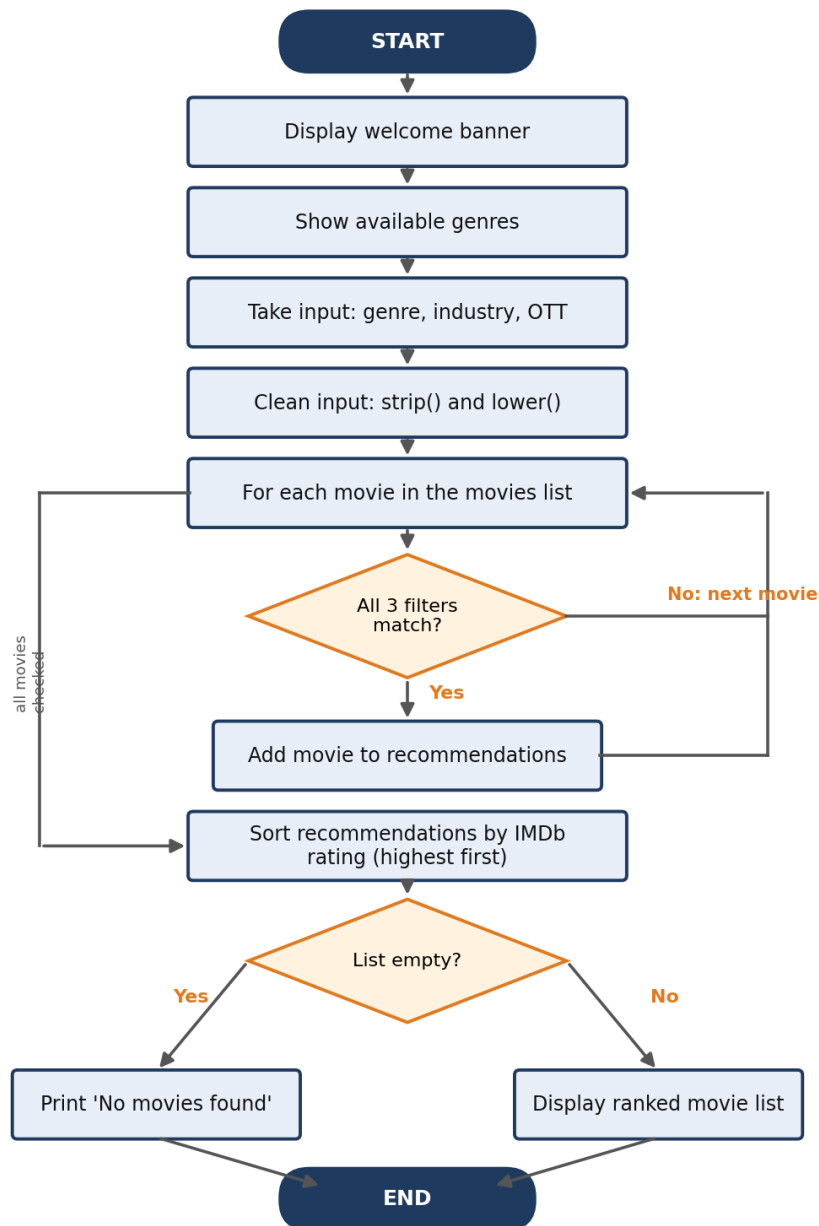


Figure 3: Workflow (flowchart) of the recommendation process

7.3 Sequence Diagram

The sequence diagram shows the order of interactions between the user, the main script, the `recommend_movies()` function, the movie list and the `show_movies()` function.

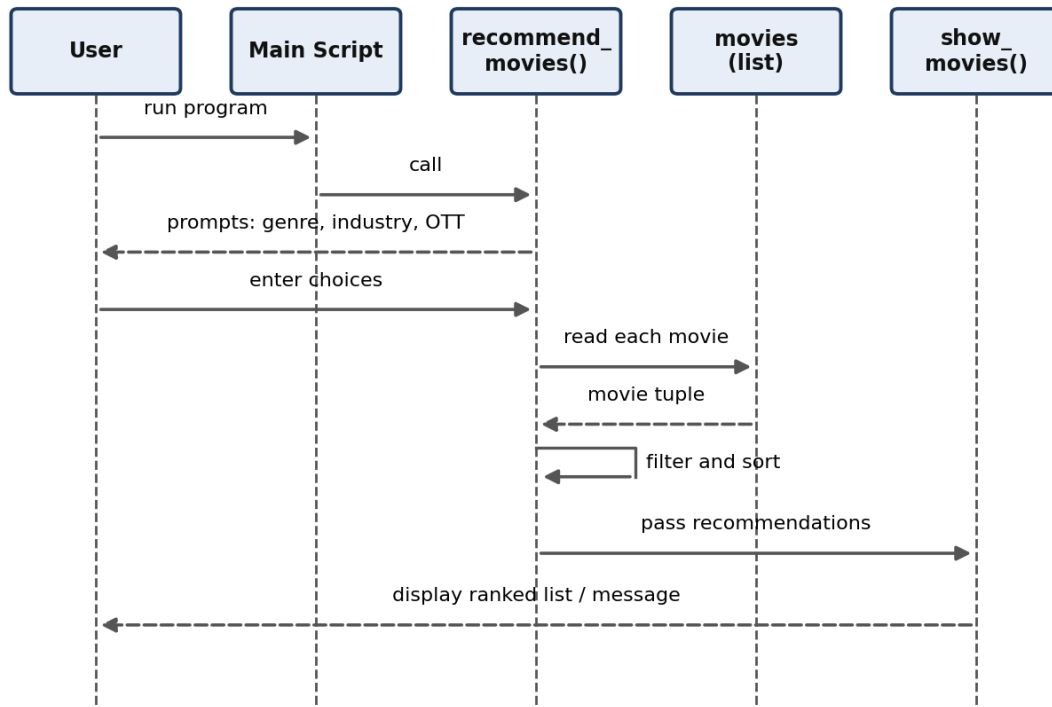


Figure 4: Sequence diagram

7.4 Class / Component Diagram

The program does not define classes; it is organised as functions and a data list. The component diagram below therefore shows the main program components and how they depend on each other.

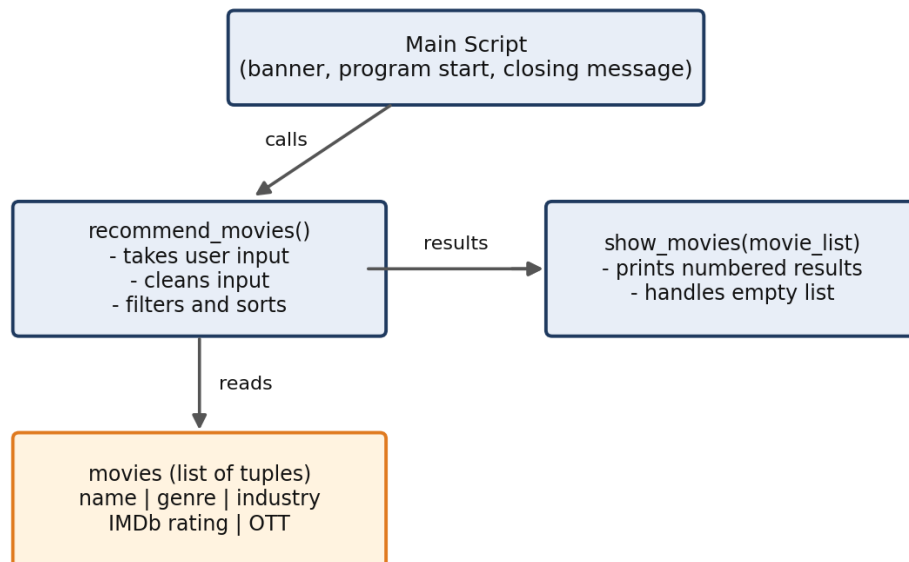


Figure 5: Component diagram

7.5 ER Diagram

Not applicable. CineMatch does not use a database or file storage. All movie data lives in an in-memory Python list, so no entity-relationship model is required. If storage is added in future (for example a SQLite database), a single *Movie* entity with the attributes name, genre, industry, IMDb rating and OTT platform would be the natural starting point.

8. Design Decisions and Rationale

Decision	Rationale
Store movies as tuples inside a list	Tuples are simple, fixed-size records suited to read-only data; a list makes it easy to loop over all movies.
Console interface	Keeps the focus on program logic, needs no extra libraries and runs anywhere Python is installed.
'Any' keyword for every filter	Gives flexibility: users can combine specific and open choices without extra menus.
Convert input to lower case and strip spaces	Makes matching forgiving; 'DRAMA', ' drama ' and 'Drama' all behave the same.
Manual sorting with nested loops instead of sort()	Shows how sorting works using only basic loops and conditions; suitable for a beginner-level project.
Separate functions for input/logic and output	<code>recommend_movies()</code> and <code>show_movies()</code> each have one job, which improves readability and testing.

Decision	Rationale
In-memory data instead of a database	The dataset is small and fixed, so a database would add complexity without benefit.

9. Implementation Details

The program is a single Python file with two functions and a small main section.

Data. The movies list holds all records in the form (name, genre, industry, IMDb rating, OTT platform).

recommend_movies(). It prints the available genres, reads the three inputs with `input()`, cleans them using `strip()` and `lower()`, and loops through the movie list. A movie is added to the result only when all three conditions are true:

```
genre_match    = genre == "any" or movie_genre == genre
industry_match = industry == "any" or movie_industry == industry
ott_match      = ott == "any" or movie_ott == ott

if genre_match and industry_match and ott_match:
    recommendations.append(movie)
```

The selected movies are then arranged from the highest to the lowest IMDb rating with a nested loop that swaps two entries whenever the earlier one has the lower rating:

```
for i in range(len(recommendations)):
    for j in range(i + 1, len(recommendations)):
        if recommendations[i][3] < recommendations[j][3]:
            temp = recommendations[i]
            recommendations[i] = recommendations[j]
            recommendations[j] = temp
```

show_movies(). If the list is empty it prints 'No movies found for your choices.' Otherwise it prints a numbered list showing the name, genre, industry, IMDb rating and OTT platform of each movie.

Main section. It prints the banner, calls `recommend_movies()` and finally prints a thank-you message.

Complexity. Filtering checks each of the n movies once, so it takes $O(n)$ time. The nested-loop sort on m matching movies takes $O(m^2)$ time, which is perfectly acceptable for a list of about 100 movies.

10. Screenshots / Results

The screenshots below show real output of the program for three different sets of choices.

Example 1 - Thriller / Bollywood / Prime Video. Four movies match and are listed with the highest rating first.



```
Terminal - python cinematch.py

=====
CINEMATCH MOVIE RECOMMENDER
=====

--- CineMatch Movie Recommendation ---

Available Genres:
Action, Comedy, Crime, Drama, Fantasy, Horror, Romance,
Sci-Fi, Thriller, War, Western

Enter genre (or type Any): Thriller
Enter industry Hollywood/Bollywood (or type Any): Bollywood
Enter OTT Netflix/Prime Video/Disney+ (or type Any): Prime Video

Recommended Movies
-----
1 . Andhadhun
   Genre: Thriller
   Industry: Bollywood
   IMDb: 8.2
   OTT: Prime Video

2 . Drishyam
   Genre: Thriller
   Industry: Bollywood
   IMDb: 8.2
   OTT: Prime Video

3 . Drishyam 2
   Genre: Thriller
   Industry: Bollywood
   IMDb: 8.2
   OTT: Prime Video

4 . Kahaani 2
   Genre: Thriller
   Industry: Bollywood
   IMDb: 6.6
   OTT: Prime Video

Thank you for using CineMatch!
```

Figure 6: Recommendations for Thriller, Bollywood, Prime Video

Example 2 - Action / Hollywood / Disney+. The Disney+ action titles are ranked by IMDb rating.

```
Terminal - python cinematch.py

=====
CINEMATCH MOVIE RECOMMENDER
=====

--- CineMatch Movie Recommendation ---

Available Genres:
Action, Comedy, Crime, Drama, Fantasy, Horror, Romance,
Sci-Fi, Thriller, War, Western

Enter genre (or type Any): Action
Enter industry Hollywood/Bollywood (or type Any): Hollywood
Enter OTT Netflix/Prime Video/Disney+ (or type Any): Disney+

Recommended Movies
-----
1 . Avengers: Endgame
   Genre: Action
   Industry: Hollywood
   IMDb: 8.4
   OTT: Disney+

2 . The Avengers
   Genre: Action
   Industry: Hollywood
   IMDb: 8.0
   OTT: Disney+

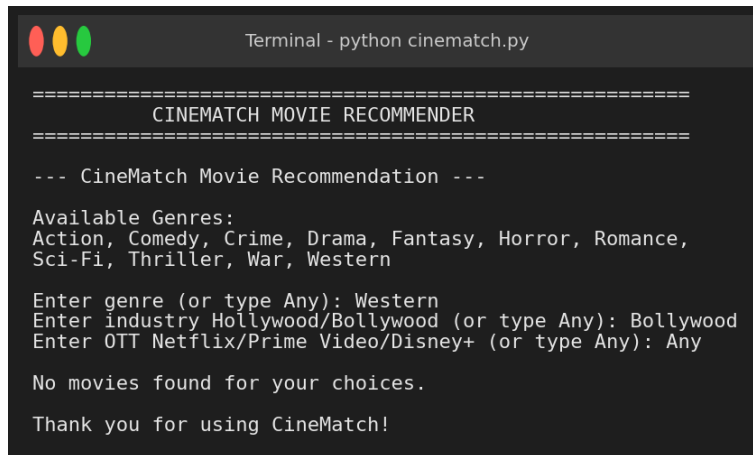
3 . Guardians of the Galaxy
   Genre: Action
   Industry: Hollywood
   IMDb: 8.0
   OTT: Disney+

4 . Iron Man
   Genre: Action
   Industry: Hollywood
   IMDb: 7.9
   OTT: Disney+

Thank you for using CineMatch!
```

Figure 7: Recommendations for Action, Hollywood, Disney+

Example 3 - Western / Bollywood / Any. No Bollywood Western exists in the list, so the program shows a friendly message instead of failing.



```

Terminal - python cinematch.py

=====
CINEMATCH MOVIE RECOMMENDER
=====

--- CineMatch Movie Recommendation ---

Available Genres:
Action, Comedy, Crime, Drama, Fantasy, Horror, Romance,
Sci-Fi, Thriller, War, Western

Enter genre (or type Any): Western
Enter industry Hollywood/Bollywood (or type Any): Bollywood
Enter OTT Netflix/Prime Video/Disney+ (or type Any): Any

No movies found for your choices.

Thank you for using CineMatch!

```

Figure 8: No matching movies found

11. Testing Approach

Testing was done manually (black-box testing): the program was run with a variety of inputs and the output was compared with the result expected from the dataset. The tests covered normal combinations, the 'Any' option, mixed upper/lower case with extra spaces, and combinations with no matching movie.

#	Genre	Industry	OTT	Matches	Top result (rating)	Outcome
1	Comedy	Bollywood	Netflix	9	3 Idiots (8.4)	Pass
2	Sci-Fi	Hollywood	Prime Video	6	Inception (8.8)	Pass
3	Any	Any	Any	100	The Shawshank Redemption (9.3)	Pass
4	drama	BOLLYWOOD	netflix	13	Taare Zameen Par (8.3)	Pass
5	Western	Bollywood	Any	0	No movies found message	Pass
6	Horror	Any	Disney+	0	No movies found message	Pass
7	Musical	Any	Any	0	No movies found message	Pass

What was checked in every test

- Movies appear in order from the highest to the lowest IMDb rating.
- Typing **Any** correctly skips the corresponding filter.
- Upper/lower case and extra spaces do not change the result.
- A combination with no matching movie produces the 'No movies found' message instead of an error.
- Every displayed field matches the stored data of the movie.

Note: the program does not validate the spelling of genres, so an unknown genre such as 'Musical' simply returns no matches (test 7).

12. Challenges Faced

Challenge	How it was handled
Sorting without the built-in sort()	Wrote a nested-loop sort that swaps entries when the earlier rating is lower, keeping the code beginner-friendly.
Inconsistent user input (capital letters, extra spaces)	Applied strip() and lower() to every input and to the stored values before comparing.
Combining three optional filters	Used three separate match variables joined with 'and', each allowing the keyword 'Any'.
Handling empty results	Checked the length of the result list and printed a clear message when it is zero.
Keeping the movie data consistent	Stored every record in the same five-field tuple format and used one accurate rating and platform per movie.
Static data	Ratings and OTT availability change over time, so the built-in data is only a snapshot and must be updated manually.

13. Learnings and Key Takeaways

- Practical use of lists, tuples, loops, conditions and functions in a complete program.
- How to filter data using several conditions combined with logical operators.
- How a simple sorting algorithm works and how its cost grows with the data size.
- Why input cleaning (strip and lower) matters for user-friendly programs.
- The value of splitting code into functions with a single responsibility.
- How to design, document and test a project using UML diagrams and a test table.

14. Future Enhancements

- Add a minimum IMDb rating filter and allow search by movie name.
- Include more movies and more industries (South Indian, Korean and others).
- Load movie data from a CSV or JSON file, or a database, instead of hard-coding it.
- Validate genre, industry and OTT inputs and ask the user to retry on mistakes.
- Replace the manual sort with Python's built-in sorted() for larger datasets.
- Add a graphical or web interface and personalised recommendations based on watch history.

15. References

- Python Software Foundation, *The Python Tutorial* - <https://docs.python.org/3/tutorial/>
- Python Software Foundation, *Python Standard Library documentation* - <https://docs.python.org/3/library/>
- IMDb, movie ratings used in the dataset - <https://www.imdb.com>
- Netflix, Prime Video and Disney+ platform catalogues for the OTT information - <https://www.netflix.com>, <https://www.primevideo.com>, <https://www.disneyplus.com>
- UML Diagrams reference for use case, sequence and component diagrams - <https://www.uml-diagrams.org>
- Course material and project guidelines provided on the Vityarthi portal.